

Лабораторная работа №8

Оптимизация

Чемоданова Ангелина Александровна

2 ноября 2025

Российский университет дружбы народов имени Патриса Лумумбы, Москва, Россия

- Чемоданова Ангелина Александровна
- Студентка НФИбд-02-22
- Российский университет дружбы народов имени Патриса Лумумбы
- 1132226443@pfur.ru
- <https://github.com/aachemodanova>



Цель работы

Основная цель работы — освоить пакеты Julia для решения задач оптимизации.

Задание

1. Используя Jupyter Lab, повторите примеры.
2. Выполните задания для самостоятельной работы.

Выполнение лабораторной работы

```
import Pkg
✓ 0.0s Julia

using JuMP
using GLPK
✓ 4.5s Julia

# Определение объекта модели с именем model:
model = Model{GLPK.Optimizer}
✓ 0.0s Julia

A JuMP Model
├ solver: GLPK
├ objective_sense: FEASIBILITY_SENSE
├ num_variables: 0
├ num_constraints: 0
└ Names registered in the model: none

# Определение переменных x, y и граничных условий для них:
@variable(model, x >= 0)
@variable(model, y >= 0);
✓ 0.0s Julia

# Определение ограничений модели:
@constraint(model, 6x + 8y >= 180)
@constraint(model, 7x + 12y >= 120);
✓ 1.5s Julia
```

Рис. 1: Линейное программирование

Выполнение лабораторной работы

<pre># Определение целевой функции: @objective(model, Min, 12x + 20y)</pre>	Julia
✓ 1.8s	
$12x + 20y$	
<pre># Вызов функции оптимизации: optimize!(model)</pre>	Julia
✓ 3.7s	
<pre># Определение причины завершения работы оптимизатора: termination_status(model)</pre>	Julia
✓ 2.2s	
OPTIMAL::TerminationStatusCode = 1	
<pre># Демонстрация первичных результирующих значений переменных x и y: @show value(x); @show value(y); # Демонстрация результата оптимизации: @show objective_value(model);</pre>	Julia
✓ 9.3s	
value(x) = 14.999999999999993 value(y) = 1.2500000000000047 objective_value(model) = 205.0	

Рис. 2: Линейное программирование

Выполнение лабораторной работы

8.2.2

```
# Определение объекта модели с именем vector_model:  
vector_model = Model{GLPK.Optimizer}
```

✓ 0.0s

Julia

```
A JuMP Model  
└ solver: GLPK  
└ objective_sense: FEASIBILITY_SENSE  
└ num_variables: 0  
└ num_constraints: 0  
└ Names registered in the model: none
```

```
# Определение начальных данных:  
A = [ 1 1 9 5;  
      3 5 0 8;  
      2 0 6 13]  
b = [7; 3; 5]  
c = [1; 3; 5; 2]
```

✓ 3.3s

Julia

```
4-element Vector{Int64}:  
 1  
 3  
 5  
 2
```

```
# Определение вектора переменных:  
@variable(vector_model, x[1:4] >= 0)
```

✓ 2.9s

Julia

```
4-element Vector{VariableRef}:  
 x[1]  
 x[2]  
 x[3]  
 x[4]
```

Рис. 3: Векторизованные ограничения и целевая функция оптимизации

Выполнение лабораторной работы

```
# Определение ограничений модели:
@constraint(vector_model, A * x .== b)
✓ 3.9s Julia

3-element Vector{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}, MathOptInterface.EqualTo{Float64}}, ScalarShape}}:
 x[1] + x[2] + 9 x[3] + 5 x[4] == 7
 3 x[1] + 5 x[2] + 8 x[4] == 3
 2 x[1] + 6 x[3] + 13 x[4] == 5

# Определение целевой функции:
@objective(vector_model, Min, c' * x)
✓ 0.1s Julia

 $x_1 + 3x_2 + 5x_3 + 2x_4$ 

# Вызов функции оптимизации:
optimize!(vector_model)
✓ 0.1s Julia

# Определение причины завершения работы оптимизатора:
termination_status(vector_model)
✓ 0.0s Julia

OPTIMAL::TerminationStatusCode = 1

# Демонстрация результата оптимизации:
@show objective_value(vector_model);
✓ 0.0s Julia

objective_value(vector_model) = 4.9230769230769225
```

Рис. 4: Векторизованные ограничения и целевая функция оптимизации

Выполнение лабораторной работы

8.2.3

Контейнер для хранения данных об ограничениях на количество потребляемых калорий, белков, жиров и соли:

```
category_data = JuMP.Containers.DenseAxisArray{ [1800 2200;  
91 Inf;  
0 65;  
0 1779],  
["calories", "protein", "fat", "sodium"],  
["min", "max"]}
```

✓ 3.8s

Julia

2-dimensional DenseAxisArray{Float64,2,...} with index sets:

Dimension 1, ["calories", "protein", "fat", "sodium"]
Dimension 2, ["min", "max"]

And data, a 4x2 Matrix{Float64}:

```
1800.0  2200.0  
91.0    Inf  
0.0     65.0  
0.0    1779.0
```

массив данных с наименованиями продуктов:

```
foods = ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
```

✓ 0.7s

Julia

9-element Vector{String}:

```
"hamburger"  
"chicken"  
"hot dog"  
"fries"  
"macaroni"  
"pizza"  
"salad"  
"milk"  
"ice cream"
```

Рис. 5: Оптимизация рациона питания

Выполнение лабораторной работы

```
# Массив стоимости продуктов:
cost = JuMP.Containers.DenseAxisArray{
  [2.49, 2.89, 1.50, 1.89, 2.09, 1.99, 2.49, 0.89, 1.59],
  foods)
✓ 0.9s
```

1-dimensional DenseAxisArray{Float64,1,...} with index sets:
Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
And data, a 9-element Vector{Float64}:
2.49
2.89
1.5
1.89
2.09
1.99
2.49
0.89
1.59

Julia

Рис. 6: Оптимизация рациона питания

Выполнение лабораторной работы

```
# Массив данных о содержании калорий, белков, жиров и соли в продуктах питания:  
food_data = JuMP.Containers.DenseAxisArray{  
[410 24 26 730;  
420 32 10 1190;  
560 20 32 1800;  
380 4 19 270;  
320 12 10 930;  
320 15 12 820;  
320 31 12 1230;  
100 8 2.5 125;  
330 8 10 180],  
foods,  
["calories", "protein", "fat", "sodium"]}
```

✓ 0.3s

Julia

```
2-dimensional DenseAxisArray{Float64,2,...} with index sets:  
Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]  
Dimension 2, ["calories", "protein", "fat", "sodium"]  
And data, a 9x4 Matrix{Float64}:  
410.0 24.0 26.0 730.0  
420.0 32.0 10.0 1190.0  
560.0 20.0 32.0 1800.0  
380.0 4.0 19.0 270.0  
320.0 12.0 10.0 930.0  
320.0 15.0 12.0 820.0  
320.0 31.0 12.0 1230.0  
100.0 8.0 2.5 125.0  
330.0 8.0 10.0 180.0
```

Рис. 7: Оптимизация рациона питания

Выполнение лабораторной работы

```
# Определение объекта модели с именем model:  
model = Model{GLPK.Optimizer}
```

✓ 0.0s

Julia

```
A JuMP Model  
├ solver: GLPK  
├ objective_sense: FEASIBILITY_SENSE  
├ num_variables: 0  
├ num_constraints: 0  
└ Names registered in the model: none
```

```
# Определим массив:
```

```
categories = ["calories", "protein", "fat", "sodium"]
```

✓ 0.0s

Julia

```
4-element Vector{String}:  
"calories"  
"protein"  
"fat"  
"sodium"
```

```
# Определение переменных:
```

```
@variables(model, begin  
    category_data[c, "min"] <= nutrition[c = categories] <= category_data[c, "max"]  
    # Сколько покупать продуктов:  
    buy[foods] >= 0  
end)
```

✓ 2.0s

Julia

```
(1-dimensional DenseAxisArray{VariableRef,1,...} with index sets:  
 Dimension 1, ["calories", "protein", "fat", "sodium"]  
And data, a 4-element Vector{VariableRef}:  
 nutrition[calories]  
 nutrition[protein]  
 nutrition[fat]  
 nutrition[sodium], 1-dimensional DenseAxisArray{VariableRef,1,...} with index sets:  
 Dimension 1, ["hamburger", "chicken", "hot dog", "fries", "macaroni", "pizza", "salad", "milk", "ice cream"]
```

Рис. 8: Оптимизация рациона питания

Выполнение лабораторной работы

```
# Определение целевой функции:
@objective(model, Min, sum(cost[f] * buy[f] for f in foods))
✓ 0.3s Julia

2.49buyhamburger + 2.89buychicken + 1.5buyhotdog + 1.89buyfries + 2.09buymacaroni + 1.99buypizza + 2.49buy salad + 0.89buy milk + 1.59buyicecream

# Определение ограничений модели:
@constraint(model, [c in categories],
sum(food_data[f, c] * buy[f] for f in foods) == nutrition[c])
✓ 1.1s Julia

1-dimensional DenseAxisArray{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}, MathOptInterface.EqualTo{Float64}}}, Scala
Dimension 1, ["calories", "protein", "fat", "sodium"]}
And data, a 4-element Vector{ConstraintRef{Model, MathOptInterface.ConstraintIndex{MathOptInterface.ScalarAffineFunction{Float64}, MathOptInterface.EqualTo{Float64}}}, Scala
-nutrition[calories] + 410 buy[hamburger] + 420 buy[chicken] + 560 buy[hot dog] + 380 buy[fries] + 320 buy[macaroni] + 320 buy[pizza] + 320 buy[salad] + 100 buy[milk] + 33
-nutrition[protein] + 24 buy[hamburger] + 32 buy[chicken] + 20 buy[hot dog] + 4 buy[fries] + 12 buy[macaroni] + 15 buy[pizza] + 31 buy[salad] + 8 buy[milk] + 8 buy[ice cre
-nutrition[fat] + 26 buy[hamburger] + 10 buy[chicken] + 32 buy[hot dog] + 19 buy[fries] + 10 buy[macaroni] + 12 buy[pizza] + 12 buy[salad] + 2.5 buy[milk] + 10 buy[ice cre
-nutrition[sodium] + 730 buy[hamburger] + 1190 buy[chicken] + 1800 buy[hot dog] + 270 buy[fries] + 930 buy[macaroni] + 820 buy[pizza] + 1230 buy[salad] + 125 buy[milk] + 1

⏪ Generate + Code + Markdown ⏩

# Вывод функции оптимизации:
JuMP.optimize!(model)
term_status = JuMP.termination_status(model)
✓ 0.1s Julia

OPTIMAL::TerminationStatusCode = 1
```

Рис. 9: Оптимизация рациона питания

```
hcat(buy.data, JuMP.value.(buy.data))
```

✓ 1.9s Julia

9×2 Matrix{AffExpr}:

buy[hamburger]	0.6045138888888888
buy[chicken]	0
buy[hot dog]	0
buy[fries]	0
buy[macaroni]	0
buy[pizza]	0
buy[salad]	0
buy[milk]	6.9701388888888935
buy[ice cream]	2.5913194444444441

Рис. 10: Оптимизация рациона питания

Выполнение лабораторной работы

8.2.4

```
using DelimitedFiles
using CSV
```

✓ 1.3s

Julia

Считывание данных:

```
passportdata = readlm(joinpath("passport-index-dataset", "passport-index-matrix.csv"),',')
```

✓ 2.2s

Julia

```
200×200 Matrix{Any}:
"Passport"      "Albania"      --      "Afghanistan"
"Afghanistan"    "visa required" -1
"Albania"        -1             "visa required"
"Algeria"        "visa required" "visa required"
"Andorra"        98             "visa required"
"Angola"         "visa required" --      "visa required"
"Antigua and Barbuda" 98             "visa required"
"Argentina"      98             "visa required"
"Armenia"        98             "visa required"
"Australia"      98             "visa required"
[...]
"Uruguay"        98             \      "visa required"
"Uzbekistan"     "visa required" "visa required"
"Vanuatu"        "visa required" "visa required"
"Vatican"        98             "visa required"
"Venezuela"      98             "visa required"
"Vietnam"        "visa required" --      "visa required"
"Yemen"          "visa required" "visa required"
"Zambia"         "visa required" "visa required"
"Zimbabwe"      "visa required" "visa required"
```

Рис. 11: Путешествие по миру

Выполнение лабораторной работы

```
# Задать переменные:
cntr = passportdata[2:end,1]
vf = (x -> typeof(x)==Int64 || x == "VF" || x == "VQA" ? 1 : 0).(passportdata[2:end,2:end]);
```

✓ 0.3s

Julia

```
# Определение объекта модели с именем model:
model = Model{GLPK.Optimizer}
```

✓ 0.0s

Julia

```
A JuMP Model
├ solver: GLPK
├ objective_sense: FEASIBILITY_SENSE
├ num_variables: 0
├ num_constraints: 0
└ Names registered in the model: none
```

```
# Переменные, ограничения и целевая функция:
@variable(model, pass[1:length(cntr)], Bin)
@constraint(model, [j=1:length(cntr)], sum( vf[i,j]*pass[i] for i in 1:length(cntr)) >= 1)
@objective(model, Min, sum(pass))
```

✓ 5.9s

Julia

```
pass1 + pass2 + pass3 + pass4 + pass5 + pass6 + pass7 + pass8 + pass9 + pass10 + pass11 + pass12 + pass13 + pass14 + pass15 + pass16 + pass17 + pass18 + pass19 + pass20 +  
pass21 + pass22 + pass23 + pass24 + pass25 + pass26 + pass27 + pass28 + pass29 + pass30 + [...139 terms omitted...] + pass176 + pass177 + pass178 + pass179 +  
pass175 + pass176 + pass177 + pass178 + pass179 + pass180 + pass181 + pass182 + pass183 + pass184 + pass185 + pass186 + pass187 + pass188 + pass189 + pass190 + pass191 + pass192 +  
pass193 + pass194 + pass195 + pass196 + pass197 + pass198 + pass199
```

Рис. 12: Путешествие по миру

Выполнение лабораторной работы

```
# Вызов функции оптимизации:
JuMP.optimize!(model)
termination_status(model)
✓ 0.0s Julia

OPTIMAL::TerminationStatusCode = 1

# Просмотр результата:
print(JuMP.objective_value(model), " passports: ", join(cnttr[findall(JuMP.value.(pass) .== 1)], ", "))
✓ 0.5s Julia

63.0 passports: Afghanistan, Andorra, Argentina, Australia, Azerbaijan, Bahrain, Brunei, Cambodia, Cameroon, Canada, Chile, Colombia, Comoros, DR Congo, Djibouti, Equatoria
```

Рис. 13: Путешествие по миру

Выполнение лабораторной работы

```
8.2.5

using DataFrames
using XLSX
using Plots
pyplot()
using Convex
using SCS
using Statistics

✓ 27.8s Julia

# Считываем данные и размещаем их во фрейм:
T = DataFrame(XLSX.readtable("stock_prices.xlsx", "Sheet2"))

✓ 8.3s Julia

13×3 DataFrame
Row  MSFT  FB  AAPL
Any  Any  Any
1   101.93 137.95 148.26
2   102.8  143.8  152.29
3   107.71 150.04 156.82
4   107.17 149.01 157.76
5   102.78 165.71 166.52
6   105.67 167.33 170.41
7   108.22 162.5  170.42
8   110.97 161.89 172.97
9   112.53 162.28 174.97
10  110.51 169.6  172.91
11  115.91 165.98 186.12
12  117.05 164.34 191.05
13  117.94 166.69 189.95
```

Рис. 14: Портфельные инвестиции

Выполнение лабораторной работы

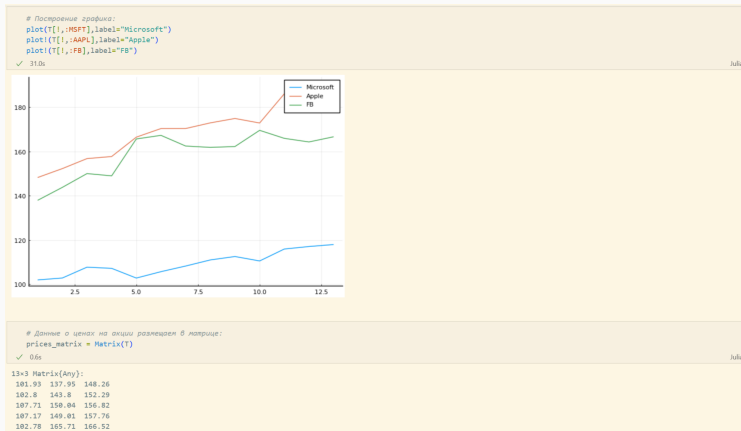


Рис. 15: Портфельные инвестиции

Выполнение лабораторной работы

```
# Вычисление матрицы доходности за период времени:
M1 = prices_matrix[1:end-1,1]
M2 = prices_matrix[2:end,:]
# Матрица доходности:
R = (M2.-M1)./M1

✓ 1.8s Julia

12×3 Matrix{Float64}:
 0.00853527  0.0424067  0.027182
 0.0477626  0.0433936  0.0297459
-0.00501346 -0.00686484  0.00599413
-0.040963   0.112073   0.0555274
 0.0281183  0.00977611  0.0233606
 0.0241317 -0.0288651  5.8682e-5
 0.0254112 -0.00375385  0.014963
 0.0140579  0.00240904  0.0115627
-0.0179508  0.0451072  -0.0117734
 0.0488644 -0.0213443  0.0763981
 0.00983522 -0.00988071  0.0264883
 0.00760359  0.0142996  -0.00575766

# Матрица рисков:
risk_matrix = cov(R)

✓ 1.0s Julia

3×3 Matrix{Float64}:
 0.000659383 -0.000630653  0.000139112
-0.000630653  0.00152162  0.000192288
 0.000139112  0.000192288  0.000635503

# Проверка положительной определённости матрицы рисков:
isposdef(risk_matrix)

✓ 1.1s Julia

true
```

Рис. 16: Портфельные инвестиции

Выполнение лабораторной работы

```
# Доход от каждой из компаний:
r = mean(R,dims=1)[:]
✓ 1.3s Julia

3-element Vector{Float64}:
 0.012532748705136572
 0.016563036855293173
 0.02114580465503291

# Вектор инвестиций:
x = Variable{length(r)}
✓ 0.8s Julia

Variable
size: (3, 1)
sign: real
vexity: affine
id: 151.465

# Объект модели:
problem = minimize(Convex.quadform(x,risk_matrix),[sum(x)==1;r'*x>=0.02;x.>=0])
✓ 26.1s Julia

Problem statistics
problem is DCP      : true
number of variables : 1 (3 scalar elements)
number of constraints : 5 (5 scalar elements)
number of coefficients : 19
number of atoms      : 14

Solution summary
termination status : OPTIMIZE_NOT_CALLED
primal status      : NO_SOLUTION
dual status         : NO_SOLUTION

Expression graph
minimize
```

Рис. 17: Портфельные инвестиции

Выполнение лабораторной работы

```
# Находим решение:
solve!(problem, SCS.Optimizer)

✓ 34.4s Julia
```

Info: [Convex.jl] Compilation finished: 25.81 seconds, 810.603 MiB of memory allocated
@ Convex <C:\Users\angelina\Julia\packages\Convex\TPPoRVars\solution.jl:107>

SCS v3.2.9 - Splitting Conic Solver
(c) Brendan O'Donoghue, Stanford University, 2012

problem: variables n: 5, constraints m: 12
cones: z: primal zero / dual free vars: 1
l: linear vars: 4
q: soc vars: 7, qsize: 2
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
max_iters: 100000, normalize: 1, rho_x: 1.00e-06
acceleration_lookback: 10, acceleration_interval: 10
compiled with openmp parallelization enabled
lin-sys: sparse-direct-and-qdldl
nnz(A): 22, nnz(P): 0

iter | pri res | dua res | gap | obj | scale | time (s)

0 | 1.45e+01 | 1.00e+00 | 2.01e+01 | -9.96e+00 | 1.00e-01 | 9.30e-03
75 | 1.70e-04 | 3.73e-05 | 5.25e-06 | 4.62e-04 | 1.00e-01 | 9.79e-03

status: solved
timings: total: 9.80e-03s = setup: 1.75e-03s + solve: 8.05e-03s
lin-sys: 2.59e-04s, cones: 5.56e-05s, accel: 8.80e-06s

objective = 0.000462

Problem statistics
problem is DCP : true
number of variables : 1 (3 scalar elements)
number of constraints : 5 (5 scalar elements)
number of coefficients : 19
number of atoms : 14

Рис. 18: Портфельные инвестиции

<pre>sum(x.value)</pre> ✓ 0.3s	Julia
1.000002434677944	
<pre>r'*x.value</pre> ✓ 1.7s	Julia
1x1 adjoint(::Vector{Float64}) with eltype Float64: 0.019947233108531883	
<pre>x.value .* 1000</pre> ✓ 0.1s	Julia
3x1 Matrix{Float64}: 77.40709795031023 116.06771003983785 806.5276266877958	

Рис. 19: Портфельные инвестиции

8.2.6

```
using Images
using Convex
using SCS
```

✓ 0.0s

Julia

Считывание исходного изображения:

```
Kref = load("kham-small.jpg")
```

✓ 6.3s

Julia



Рис. 20: Портфельные инвестиции

Выполнение лабораторной работы

```
K = copy(Kref)
p = prod(size(K))
missingids = rand(1:p,400)
K[missingids] .= RGBX{Nof0}(0.0,0.0,0.0)
K
Gray.(K)
```

✓ 4.7s

Julia



```
# Плотность шума:
Y = Float64.(Gray.(K));
```

✓ 0.3s

Julia

```
correctids = findall(Y[:,].!=0)
X = Convex.Variable{size(Y)}
problem = minimize(nuclearnorm(X))
problem.constraints += X[correctids].==Y[correctids]
```

✓ 6.0s

Julia

Warning: Concatenating collections of constraints together with '+' or '+=` to produce a new list of constraints is deprecated. Instead, use `vcat` to concatenate collect
@ Convex [C:\Users\angelina\julia\packages\Convex\TPPoR\src\deprecations.jl:128](https://github.com/JuliaOpt/Convex.jl/blob/master/src/deprecations.jl#L128)

Рис. 21: Восстановление изображения

Выполнение лабораторной работы

```
# Находим решение:
solve!(problem, SCS.Optimizer)
✓ 22.6s
```

SCS v3.2.9 - Splitting Conic Solver
(c) Brendan O'Donoghue, Stanford University, 2012

problem: variables n: 88090, constraints m: 159780
cones: z: primal zero / dual free vars: 79690
nuc: nuclear vars: 88090, nucsize: 1
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infvars: 1.0e-07
alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
max_iters: 100000, normalize: 1, rho_x: 1.00e-06
acceleration_lookback: 10, acceleration_interval: 10
compiled with openmp parallelization enabled
lin-sys: sparse-direct-and-qdldl
nnz(A): 159780, nnz(P): 0

iter	pr	res	dual res	gap	obj	scale	time (s)
0		1.27e+02	1.17e+01	3.09e+03	1.53e+03	1.00e-01	3.46e-01
158		3.91e-03	6.42e-05	1.22e-04	4.45e+02	9.31e-03	1.50e+01

status: solved
timings: total: 1.50e+01s = setup: 1.71e-01s + solve: 1.48e+01s
lin-sys: 7.54e-01s, cones: 1.37e+01s, accel: 6.37e-02s

objective = 445.300354

Info: [Convex.jl] Compilation finished: 5.69 seconds, 270.493 MiB of memory allocated
@ Convex C:\Users\angelina\julia\packages\Convex\1PPoR\src\solution.jl:107

Problem statistics
problem is DCP : true
number of variables : 1 (88_089 scalar elements)
number of constraints : 1 (79_690 scalar elements)
number of coefficients : 79_690
number of atoms : 3

Рис. 22: Восстановление изображения



Рис. 23: Восстановление изображения

Выполнение лабораторной работы

8.4.1

```
model = Model{GLPK.Optimizer}
```

✓ 0.0s

Julia

```
A JuMP Model
├ solver: GLPK
├ objective_sense: FEASIBILITY_SENSE
├ num_variables: 0
├ num_constraints: 0
└ Names registered in the model: none
```

[Generate](#) [+ Code](#) [+ Markdown](#)

```
@variable(model, 0<=x1<=10)
@variable(model, x2>=0)
@variable(model, x3>=0)

@constraint(model, -x1+x2+3x3 <= -5)
@constraint(model, x1+3x2-7x3 <= 10)

@objective(model, Max, x1+2x2+5x3)
```

✓ 0.0s

Julia

$x1 + 2x2 + 5x3$

```
optimize!(model)
```

```
termination_status(model)
```

✓ 0.0s

Julia

OPTIMAL::TerminationStatusCode = 1

Рис. 24: Задание 1. Линейное программирование

```
@show value(x1);  
@show value(x2);  
@show value(x3);  
  
@show objective_value(model);  
✓ 0.0s Julia  
  
value(x1) = 10.0  
value(x2) = 2.1875  
value(x3) = 0.9375  
objective_value(model) = 19.8625
```

Рис. 25: Задание 1. Линейное программирование

Выполнение лабораторной работы

8.4.2

```
vector_model = Model{GLPK.Optimizer}
A = [-1 1 3; 1 3 -7]
b = [-5; 10]
c = [1; 2; 5]
```

✓ 0.0s

Julia

```
3-element Vector{Int64}:
 1
 2
 5
```

```
@variables(vector_model, begin
    X[1:3]>=0
end)
```

✓ 0.1s

Julia

```
(VariableRef{X[1], X[2], X[3]},)
```

```
@constraint(vector_model, A * X .<= b)
@constraint(vector_model, X[1] <= 10)
@objective(vector_model, Min, c' * X)
```

✓ 0.0s

Julia

$X_1 + 2X_2 + 5X_3$

[Generate](#) [+ Code](#) [+ Markdown](#)

```
optimize!(vector_model)
termination_status(model)
```

✓ 0.0s

Julia

OPTIMAL::TerminationStatusCode = 1

Рис. 26: Задание 2. Линейное программирование. Использование массивов

```
@show value(X1);  
@show value(X2);  
@show value(X3);  
✓ 0.0s Julia  
value(x1) = 18.0  
value(x2) = 2.1875  
value(x3) = 0.9375  
  
@show objective_value(vector_model);  
✓ 0.0s Julia  
objective_value(vector_model) = 5.0
```

Рис. 27: Задание 2. Линейное программирование. Использование массивов

Выполнение лабораторной работы

8.4.3

```
n = 3
m = 4
A = rand(1:100, m, n);
b = rand(1:100, m);
```

✓ 0.0s

Julia

```
x = Variable{n}
problem = minimize(sumsquares(A*x-b))
constraints = [x>=0]
solve!(problem, SCS.Optimizer)
```

✓ 0.0s

Julia

```
-----
      SCS v3.2.9 - Splitting Conic Solver
      (c) Brendan O'Donoghue, Stanford University, 2012
-----
problem: variables n: 4, constraints m: 6
cones:   q: soc vars: 6, qsize: 1
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
          alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
          max_iters: 100000, normalize: 1, rho_x: 1.00e-06
          acceleration_lookback: 10, acceleration_interval: 10
          compiled with openmp parallelization enabled
lin-sys: sparse-direct-amd-qdldl
          nnz(A): 14, nnz(P): 0
-----
iter | pri res | dua res | gap | obj | scale | time (s)
-----
0 | 1.52e+03 | 1.00e+00 | 2.15e+03 | -1.07e+03 | 1.00e-01 | 4.03e-04
250 | 1.29e+00 | 1.41e-02 | 1.20e+00 | 2.10e+02 | 2.42e+00 | 1.96e-03
350 | 3.00e-05 | 8.75e-07 | 7.53e-06 | 2.10e+02 | 2.42e+00 | 2.40e-03
-----
status: solved
timings: total: 2.40e-03s = setup: 1.24e-04s + solve: 2.28e-03s
          lin-sys: 9.97e-05s, cones: 5.52e-05s, accel: 8.72e-05s
```

Рис. 28: Задание 3. Выпуклое программирование

problem.optval		
✓	0.0s	Julia
209.99693146573885		

Рис. 29: Задание 3. Выпуклое программирование

Выполнение лабораторной работы

```
8.4.4

using Random
✓ 0.0s Julia

priorities = [shuffle([1, 2, 3, 10000, 10000]) for _ in 1:1000]
priorities = hcat(priorities...)
priority_weights = sum(priorities, dims = 1)
✓ 0.1s Julia

1x5 Matrix{Int64}:
3871195  4221148  3971229  4031189  3911239

modell = Model{GLPK.Optimizer};
✓ 0.0s Julia

@variable(modell, 100<=t1<=250)
@variable(modell, 100<=t2<=250)
@variable(modell, t3 == 220)
@variable(modell, 100<=t4<=250)
@variable(modell, 100<=t5<=250);
✓ 0.0s Julia

@constraint(modell, t1+t2+t3+t4+t5==1000)
@objective(modell, Min, t1 * priority_weights[1] +
t2 * priority_weights[2] +
t3 * priority_weights[3] +
t4 * priority_weights[4] +
t5 * priority_weights[5])
✓ 0.0s Julia

3871195t1 + 4221148t2 + 3971229t3 + 4031189t4 + 3911239t5
```

Рис. 30: Задание 4. Оптимальная рассадка по залам

```
optimize!(model1)
termination_status(model1)
```

✓ 0.0s Julia

OPTIMAL::TerminationStatusCode = 1

```
println("t1 = ", value(t1))
println("t2 = ", value(t2))
println("t3 = ", value(t3))
println("t4 = ", value(t4))
println("t5 = ", value(t5))
println("Значение целевой функции: ", objective_value(model1))
```

✓ 0.0s Julia

t1 = 240.0
t2 = 180.0
t3 = 220.0
t4 = 180.0
t5 = 180.0
Значение целевой функции: 3.99220886e9

Рис. 31: Задание 4. Оптимальная рассадка по залам

Выполнение лабораторной работы

8.4.5

```
coffee = ["Raf coffee", "Cappuccino"]
products = ["Grains", "Milk", "Sugar"]
costs = JuMP.Containers.DenseAxisArray{[400, 300], coffee}
coffee_data = JuMP.Containers.DenseAxisArray{
    [40 140 5;
     30 120 0],
    coffee,
    products
}
store = JuMP.Containers.DenseAxisArray{[500, 2000, 40], products}
coffee_data["Raf coffee", "Grains"]
store["Grains"]
```

✓ 0.0s

Julia

500

```
model = Model{GLPK.Optimizer}
@variable(model, buy[coffee] >= 0)

@objective(model, Max, sum(costs[c]*buy[c] for c in coffee))
@constraint(model, [p in products], sum(coffee_data[c, p]*buy[c] for c in coffee) <= store[p])

JuMP.optimize!(model)
term_status = JuMP.termination_status(model)
hcat(buy.data, JuMP.value.(buy.data))
```

✓ 0.8s

Julia

2×2 Matrix{AffExpr}:
buy[Raf coffee] 8
buy[Cappuccino] 6

Рис. 32: Задание 5. План приготовления кофе

В ходе выполнения лабораторной работы были освоены пакеты Julia для решения задач оптимизации